

Developer's Manual

StudyFlax: Time Management for Higher Education

Faculty of Natural, Mathematical, and Engineering Sciences

Department of Informatics

Team Flax-10

Aishah Ahmed

Eric Almeida

Zerun (Harry) Chen

Cate Tay (Cate) Chew

Indixora Jozal

Kausar Khan

Hamed Latif

Dian Mahmood

Binuli Nevanma Markawitage

March 2026

Table of Contents

1. **Overview**
2. **Prerequisites**
3. **Cloning the Repository**
4. **Setting Up the Development Environment with Nix**
   - 4.1 Linux and macOS
   - 4.2 Windows (via Docker Desktop)
   - 4.3 Re-entering the Nix Shell in Future Sessions
5. **Database Setup**
   - 5.1 Automated Setup via Nix (Recommended)
   - 5.2 Manual PostgreSQL Setup
6. **Running the Application**
   - 6.1 Backend (Django)
   - 6.2 Frontend (React / Vite)
7. **Running the Tests**
   - 7.1 Backend Tests and Coverage
   - 7.2 Frontend Tests and Coverage
8. **User Fixtures and Access Credentials**
9. **Deployed System**
10. **Troubleshooting**
11. **Package Summary**
    - 11.1 Backend
    - 11.2 Frontend
    - 11.3 Package Management

Chapter 1: Overview

StudyFlax is a web-based academic time management platform built for students and teachers at King's College London. It allows students to organise their schedules, track deadlines, and manage their academic workload in one place. Teachers can create modules, set deadlines, and assign work to their students, whilst the admin oversees and manages the system as a whole.

The system consists of two software components:

- **Backend** — A Django (Python 3.14) REST API backed by a PostgreSQL database.
- **Frontend** — A React (Vite) single-page application.

The deployed application is available at:

https://flax-10-production.up.railway.app/

Chapter 2: Prerequisites

The following tools must be installed before setting up the development environment. Installation of the operating system itself is assumed.

- **Nix Package Manager (v2.33.2 or later)**
  https://nixos.org/download/#download-nix
- **Git**
  https://git-scm.com/downloads
- **Docker Desktop (Windows users only)**
  https://www.docker.com/products/docker-desktop/
- **PostgreSQL (only required for manual setup without Nix)**
  https://www.postgresql.org/download/

Chapter 3: Cloning the Repository

Clone the repository and navigate into it:

```
git clone https://github.com/hoxtl/Flax-10.git
cd Flax-10
```

Chapter 4: Setting Up the Development Environment with Nix

This project uses a Nix flake to automate the full development environment setup. Upon entering the Nix shell, the following steps are performed automatically:

- A Python 3.14 virtual environment is created and activated.

- All Python dependencies are installed from `requirements.txt`.
- All Node.js (v20) dependencies are installed via npm.
- Django database migrations are applied (`makemigrations` and `migrate`).
- The database is seeded with fixture and generated data.
- The Django development server is started on `http://localhost:8000`.
- The Vite frontend development server is started.
- Backend and frontend coverage reports are generated.

4.1 Linux and macOS

1. Navigate to the root of the repository:

```
cd /path/to/Flax-10
```

2. Enter the Nix development shell:

```
nix --extra-experimental-features 'nix-command flakes' develop -c bash
```

3. **Optional — enable experimental features permanently.** Add the following line to `/etc/nix/nix.conf` to avoid passing the flags on every invocation:

```
experimental-features = nix-command flakes
```

4. On first run, the flake creates the virtual environment and installs all dependencies. On subsequent runs, it detects the existing environment and activates it. You will be returned to a `bash` terminal once setup is complete.

4.2 Windows (via Docker Desktop)

On Windows, Nix is run inside a Docker container using the official NixOS image.

1. Open the Docker Desktop terminal.
2. Navigate to the repository folder:

```
cd \path\to\Flax-10
```

3. Create a persistent Nix store volume:

```
docker volume create nix-store
```

4. Launch the NixOS container. In PowerShell, use a backtick (`` ` ``) for line continuation:

```
docker run --rm -it `
-v "${PWD}:/work" `
-v nix-store:/nix `
-w /work `
nixos/nix:latest

# Or single-line:
docker run --rm -it -v "${PWD}:/work" -v nix-store:/nix -w /work
nixos/nix:latest
```

5. Inside the NixOS container, stage the flake file:

```
git -C "/work" add "flake.nix"
```

6. Enter the Nix development shell:

```
nix --extra-experimental-features 'nix-command flakes' develop -c bash
```

7. The container is configured with `--rm` and will be destroyed upon exit. Repeat steps 4–6 for each subsequent session.

4.3 Re-entering the Nix Shell in Future Sessions

Each time you wish to return to the development environment, run the following from the root of the repository:

```
nix --extra-experimental-features 'nix-command flakes' develop -c bash
```

Chapter 5: Database Setup

5.1 Automated Setup via Nix (Recommended)

The Nix shell hook runs `makemigrations`, `migrate`, and `seed` automatically on entry. No manual database configuration is required when using Nix.

5.2 Manual PostgreSQL Setup

If you are setting up the project without Nix:

1. Install PostgreSQL and open the `psql` terminal.
2. Accept all default connection parameters (server, database, port) by pressing Enter at each prompt.
3. When prompted for a password, enter `1234` — this is the default value expected by `settings.py`.
4. Apply migrations and seed the database from within `flax-10-project-files/backend`:

```
python manage.py makemigrations
python manage.py migrate
python manage.py seed
```

Note: If you choose a different PostgreSQL password, update the `PASSWORD` field in `settings.py` accordingly.

Chapter 6: Running the Application

6.1 Backend (Django)

From within `flax-10-project-files/backend`:

```
python manage.py runserver
```

The API will be available at `http://localhost:8000`.

6.2 Frontend (React / Vite)

From within `flax-10-project-files/frontend`:

```
npm run dev
```

The frontend will be available at `http://localhost:5173` (or the port shown in the Vite terminal output).

Note: When using the Nix shell, both servers are started automatically in the background.

Chapter 7: Running the Tests

7.1 Backend Tests and Coverage

From within `flax-10-project-files/backend`:

```
python -m coverage erase
python -m coverage run manage.py test
python -m coverage report -m
python -m coverage html
```

The HTML coverage report is generated at `flax-10-project-files/backend/htmlcov/index.html`.

Note: The Nix shell runs these commands automatically on entry.

7.2 Frontend Tests and Coverage

From within `flax-10-project-files/frontend`:

```
npm run test -- --run
npm run coverage
```

Chapter 8: User Fixtures and Access Credentials

The following fixture accounts are available after seeding the database. All passwords are `1234`.

Username	Role	Password
johndoe	Student / Admin	1234
janedoe	Student	1234
amiller	Teacher	1234

charlie	Student	1234

Additional users are seeded automatically using Faker.

Django admin panel: `http://localhost:8000/admin/`

Signing in as `johndoe` with password `1234` also grants access to admin features through the main application interface.

Chapter 9: Deployed System

The production deployment is hosted on Railway:

https://flax-10-production.up.railway.app/

Chapter 10: Troubleshooting

Nix experimental features error.

If you encounter an error relating to `nix-command` or `flakes`, ensure the flags are passed as shown in Chapter 4, or add them permanently to `/etc/nix/nix.conf`.

Database connection error.

Verify that your PostgreSQL server is running and that the password in `settings.py` matches your local PostgreSQL password (default: `1234`).

Port already in use.

If `localhost:8000` or `localhost:5173` is already occupied, terminate the conflicting process or start the server on an alternative port:

```
python manage.py runserver 8080
```

Coverage module not found.

Ensure `coverage` is installed in the virtual environment:

```
pip install coverage
```

Windows Docker container exits immediately.

Ensure Docker Desktop is running and that the `nix-store` volume has been created before executing the docker run command.

Re-seeding inflates data.

The Nix shell runs `python manage.py seed` on every entry, which may duplicate seeded data. To reset the database to a clean state, run the following from within `flax-10-project-files/backend`:

```
python manage.py flush --no-input
```

```
python manage.py migrate
```

```
python manage.py seed
```

Chapter 11: Package Summary

11.1 Backend

Package	Version	Purpose
Django	5.2.0	Web framework
djangorestframework	3.16.1	REST API framework
psycopg	3.3.2	PostgreSQL driver
psycopg-binary	3.3.2	Precompiled psycopg binaries
faker	40.4.0	Fake data generation for seeding
python-dotenv	1.2.1	Environment variable management
openai	2.21.0	OpenAI API integration
requests	2.32.5	HTTP client library
coverage	latest	Test coverage reporting

11.2 Frontend

Package	Version	Purpose
react	19.2.0	UI library
react-dom	19.2.0	React DOM renderer
vite	7.2.4	Build tool and dev server
framer-motion	12.34.0	Animation library
canvas-confetti	1.9.4	Confetti animation
eslint	9.39.1	JavaScript linter

11.3 Package Management

Tool	Version	Purpose
Nix	2.33.2	Environment package management
npm	10.8.2	JavaScript package management